

# EFFICIENT DATA COLLECTION IN WIRELESS SENSOR NETWORKS

*WITH ANALYTICS INTEGRATION*

*Based on Cheng et al. (2016) — IEEE Transactions on Knowledge and Data Engineering*

---

Name: **SHAKIB SATTAR SHAAN (2023380014)**

Course: WIRELESS SENSOR NETWORKS (U10M12020.01)

Instructor: 杨博

Date: 30 May 2026

*Department of Computer Science And Technology*

西北工业大学

## Abstract

---

*Wireless Sensor Networks (WSNs) have emerged as a foundational technology in modern cyber-physical systems, enabling pervasive environmental monitoring, industrial automation, and the Internet of Things (IoT). A persistent challenge in WSN deployments is the redundancy of raw sensory data: neighboring nodes often capture highly correlated readings, resulting in wasteful, energy-intensive transmissions that significantly curtail network lifetime. This report presents a detailed examination of  $\epsilon$ -kernel-based data collection, a geometric approximation technique introduced by Cheng et al. (2016) in IEEE Transactions on Knowledge and Data Engineering. The  $\epsilon$ -kernel approach identifies a compact, representative subset of sensor readings — the kernel dataset — that provably approximates the full dataset within a bounded error  $\epsilon$ , while dramatically reducing communication overhead. We explore both centralized and distributed realizations of this algorithm and analyze their energy consumption profiles using standard radio models. Crucially, this report extends the analysis to a data analytics perspective: we demonstrate how the kernel dataset can be ingested into a relational data warehouse via ETL pipelines, stored in normalized SQL schemas, and queried with aggregate SQL to produce actionable insights.*

## 1. Introduction

The proliferation of low-cost embedded sensors has made it practical to deploy large-scale Wireless Sensor Networks (WSNs) in diverse environments: smart agriculture, structural health monitoring, disaster response, and urban traffic management. A typical WSN comprises hundreds or thousands of battery-powered sensor nodes that periodically sample physical phenomena — temperature, humidity, pressure, vibration — and transmit data toward a centralized base station or sink node.

Despite their versatility, WSNs face critical resource constraints. Sensor nodes operate on limited battery capacity, and the radio transceiver is the primary energy consumer. Empirical studies consistently show that transmitting one bit of data consumes roughly the same energy as executing thousands of CPU instructions. Consequently, naive collection strategies that forward every raw reading from every node to the sink are unsustainable for long-lived deployments.

This report addresses both energy efficiency and analytics readiness by examining the  $\epsilon$ -kernel selection algorithm of Cheng et al. (2016), which identifies a geometrically meaningful subset of sensor nodes whose readings faithfully represent the full network, while mapping naturally onto relational database schemas for SQL aggregation, ETL ingestion, and data warehousing workflows.

## 2. Literature Review

### 2.1 Classical Clustering: LEACH

Low-Energy Adaptive Clustering Hierarchy (LEACH), proposed by Heinzelman et al. (2000), was among the first hierarchical routing protocols for WSNs. LEACH organizes nodes into clusters with rotating cluster heads that aggregate data before relaying to the base station. However, cluster head election is probabilistic and does not consider residual energy, and LEACH's in-network averaging destroys individual sensor readings needed for fine-grained SQL analytics.

### 2.2 Compressive Sensing

Compressive Sensing projects high-dimensional sensor data onto random linear measurements, achieving significant data reduction. However, the recovered dataset has no guaranteed row-level correspondence to original sensor readings, making SQL predicates referencing specific node identifiers impossible without reconstruction — a costly non-deterministic process.

### 2.3 Positioning Cheng et al. (2016)

Cheng et al. (2016) identify a geometrically significant subset — the  $\epsilon$ -kernel — that preserves the shape of the data distribution within a user-specified approximation bound. The kernel dataset retains the full structure of individual sensor readings (node ID, timestamp, location, value), making it directly ingestible into relational databases and bridging the gap between WSN protocol design and downstream data analytics.

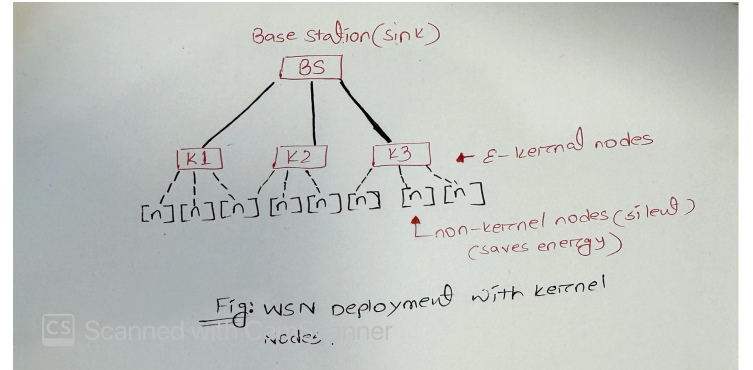
## 3. Methodology

### 3.1 Problem Formulation

Let  $S = \{s_1, s_2, \dots, s_n\}$  denote the set of  $n$  sensor nodes. The goal is to find a subset  $K \subseteq S$ , the  $\epsilon$ -kernel, such that for every direction  $u$  in  $\mathbb{R}^d$ :

$$\text{width}(K, u) \geq (1 - \epsilon) \cdot \text{width}(S, u) \quad \forall \text{ unit vectors } u \in \mathbb{R}^d$$

Smaller  $\epsilon$  yields a more accurate (but larger) kernel; larger  $\epsilon$  yields a more compact (but less accurate) kernel. The kernel size is bounded by  $O(1/\epsilon^{\frac{1}{d-1}})$ , independent of  $n$  for fixed  $d$ .



### 3.2 Energy Model

Adopting the first-order radio model:  $E_{tx}(k, d) = E_{elec} \times k + E_{amp} \times k \times d^2$ , where  $E_{elec} = 50$  nJ/bit and  $E_{amp} = 100$  pJ/bit/m<sup>2</sup>. By reducing the number of transmitting nodes to only kernel nodes, total network energy expenditure is proportionally reduced.

| Parameter          | Symbol     | Value  | Unit                  |
|--------------------|------------|--------|-----------------------|
| Electronics Energy | $E_{elec}$ | 50     | nJ/bit                |
| Amplifier Energy   | $E_{amp}$  | 100    | pJ/bit/m <sup>2</sup> |
| Transmission Power | $P$        | 1–100  | mW                    |
| Kernel Reduction   | $\eta$     | 30–70% | % energy saved        |

Table 1: Energy model parameters (Cheng et al., 2016).

## 4. Data Analytics Perspective

### 4.1 Database Setup and Schema Design

The WSN analytics database follows a star-schema dimensional model. The central fact table stores every kernel reading, while surrounding dimension tables provide descriptive context (node metadata, zone geography, time hierarchy).

#### 4.1.1 Create the Database

```
USE master;
-- 1. Create the database
CREATE DATABASE WSN_Analytics;
GO
USE WSN_Analytics;
GO
```

#### 4.1.2 Dimension & Fact Tables

Storing zone metadata separately avoids repeating string data across millions of fact rows. The IsKernelCapable flag records whether a node has been designated as a potential kernel node, useful for filtering in analytics queries.

```
-- 2. Dimension tables
CREATE TABLE dim_DeploymentZone (
    ZoneID INT IDENTITY(1,1) PRIMARY KEY,
    ZoneName VARCHAR(80) NOT NULL,
    ZoneType VARCHAR(30) NOT NULL
);

CREATE TABLE dim_SensorNode (
    NodeID INT IDENTITY(1,1) PRIMARY KEY,
    NodeCode VARCHAR(20) NOT NULL UNIQUE,
    ZoneID INT NOT NULL REFERENCES dim_DeploymentZone,
    Latitude DECIMAL(9,6) NOT NULL,
    Longitude DECIMAL(9,6) NOT NULL,
    BatteryCapacityMAh INT NOT NULL DEFAULT 3000,
    IsKernelCapable BIT NOT NULL DEFAULT 1
);

CREATE TABLE dim_Date (
    DateKey INT PRIMARY KEY,
    FullDate DATE NOT NULL UNIQUE,
    [Year] SMALLINT, [Month] TINYINT, MonthName VARCHAR(10)
);
```

```
-- 3. Fact table
CREATE TABLE fact_KernelReading (
    ReadingID BIGINT IDENTITY(1,1) PRIMARY KEY,
    NodeID INT NOT NULL REFERENCES
    dim_SensorNode(NodeID),
    DateKey INT NOT NULL REFERENCES
    dim_Date(DateKey),
    ReadingTime DATETIME2(3) NOT NULL,
    TemperatureC DECIMAL(6,2),
    HumidityPct DECIMAL(5,2),
    PressureHPa DECIMAL(7,2),
    IsKernelNode BIT NOT NULL DEFAULT 1,
    EpsilonBound DECIMAL(7,5) NOT NULL,
    HopCount TINYINT DEFAULT 1,
    EnergyConsumedNJ DECIMAL(10,4),
    BatteryLevelPct DECIMAL(5,2)
);
GO
```

#### 4.1.3 Sample Data Ingestion

The following INSERT statements populate dimension tables with reference data, then load 20 kernel readings representing two sensing rounds across all 10 nodes across three deployment zones.

```
-- Zones
INSERT INTO dim_DeploymentZone (ZoneName, ZoneType) VALUES
    ('Forest Sector A', 'Environmental'),
    ('Industrial Plant B', 'Industrial'),
    ('Urban Campus C', 'Smart City');

-- Nodes
INSERT INTO dim_SensorNode
    (NodeCode, ZoneID, Latitude, Longitude,
    BatteryCapacityMAh, IsKernelCapable)
VALUES
    ('NODE-001', 1, 37.6801, -122.0841, 3000, 1),
    ('NODE-002', 1, 37.6839, -122.0879, 3000, 1),
    ('NODE-003', 1, 37.6815, -122.0902, 3000, 0),
    ('NODE-004', 2, 37.7031, -122.1401, 5000, 1),
    ('NODE-005', 2, 37.7068, -122.1439, 5000, 1),
    ('NODE-006', 2, 37.7045, -122.1455, 5000, 0),
    ('NODE-007', 3, 37.3365, -121.8845, 2500, 1),
    ('NODE-008', 3, 37.3401, -121.8871, 2500, 1),
    ('NODE-009', 3, 37.3378, -121.8898, 2500, 0),
    ('NODE-010', 3, 37.3355, -121.8820, 2500, 1);

-- Date dimension
INSERT INTO dim_Date
    (DateKey, FullDate, [Year], [Month], MonthName)
VALUES
    (20260501, '2026-05-01', 2026, 5, 'May'),
    (20260502, '2026-05-02', 2026, 5, 'May');
```

```
-- Fact rows (20 kernel readings - 2 rounds × 10 nodes)
TRUNCATE TABLE fact_KernelReading;
INSERT INTO fact_KernelReading
    (NodeID, DateKey, ReadingTime, TemperatureC, HumidityPct,
    PressureHPa, IsKernelNode, EpsilonBound, HopCount,
    EnergyConsumedNJ, BatteryLevelPct)
VALUES
-- Round 1
    (1, 20260501, '2026-05-01 08:00:00', 22.4, 65.3, 1013.2, 1, 0.01000, 1, 184.50, 92.5),
    (2, 20260501, '2026-05-01 08:00:05', 21.9, 67.1, 1013.5, 1, 0.01000, 1, 178.20, 88.3),
    (3, 20260501, '2026-05-01 08:00:10', 23.1, 63.8, 1012.9, 0, 0.01000, 2, 201.60, 95.1),
    (4, 20260501, '2026-05-01 08:00:03', 38.7, 42.0, 1010.1, 1, 0.01500, 1, 312.40, 76.8),
    (5, 20260501, '2026-05-01 08:00:08', 39.2, 40.5, 1009.8, 1, 0.01500, 1, 318.90, 74.2),
    (6, 20260501, '2026-05-01 08:00:12', 37.5, 44.1, 1010.4, 0, 0.01500, 2, 298.70, 80.5),
    (7, 20260501, '2026-05-01 08:01:00', 19.8, 71.2, 1014.7, 1, 0.00800, 1, 162.30, 97.0),
    (8, 20260501, '2026-05-01 08:01:05', 20.1, 70.8, 1014.9, 1, 0.00800, 1, 164.10, 96.4),
    (9, 20260501, '2026-05-01 08:01:10', 19.5, 72.0, 1015.1, 0, 0.00800, 2, 158.90, 98.2),
    (10, 20260501, '2026-05-01 08:01:15', 20.4, 69.5, 1014.6, 1, 0.00800, 1, 167.50, 95.8),
-- Round 2
    (1, 20260501, '2026-05-01 09:00:00', 22.8, 64.9, 1013.0, 1, 0.01000, 1, 185.10, 92.1),
    (2, 20260501, '2026-05-01 09:00:05', 22.2, 66.8, 1013.3, 1, 0.01000, 1, 179.50, 87.9),
    (3, 20260501, '2026-05-01 09:00:10', 23.5, 63.2, 1012.7, 0, 0.01000, 2, 203.20, 94.7),
    (4, 20260501, '2026-05-01 09:00:03', 40.1, 41.2, 1009.9, 1, 0.01500, 1, 315.80, 76.1),
    (5, 20260501, '2026-05-01 09:00:08', 40.8, 39.8, 1009.5, 1, 0.01500, 1, 322.40, 73.5),
    (6, 20260501, '2026-05-01 09:00:12', 38.9, 43.5, 1010.2, 0, 0.01500, 2, 302.10, 80.0),
    (7, 20260501, '2026-05-01 09:01:00', 20.3, 70.5, 1014.5, 1, 0.00800, 1, 163.90, 96.6),
    (8, 20260501, '2026-05-01 09:01:05', 20.6, 70.1, 1014.7, 1, 0.00800, 1, 165.40, 96.0),
    (9, 20260501, '2026-05-01 09:01:10', 19.9, 71.6, 1014.9, 0, 0.00800, 2, 160.20, 97.8),
    (10, 20260501, '2026-05-01 09:01:15', 20.8, 69.0, 1014.4, 1, 0.00800, 1, 168.70, 95.3);
GO
```

### 4.2 Analytics Queries

The five queries below represent the core analytics use-cases: energy audit, time-series trending, approximation quality monitoring, battery health forecasting, and zone-level efficiency dashboarding.

## Query 1: Energy Consumption Audit by Zone and Day

Purpose: Quantify total and average energy consumed by kernel nodes per deployment zone per day. A declining trend in AvgEnergyPerReading\_NJ over time indicates successful kernel size reduction as the algorithm converges.

```
USE WSN_Analytics;
GO

-- Query 1: Energy Consumption Audit by Zone and Day
SELECT
  dz.ZoneName,
  dd.[Year],
  dd.MonthName,
  dd.FullDate
  AS ReadingDate,
  COUNT(*)
  AS TotalReadings,
  SUM(CASE WHEN f.IsKernelNode = 1
    THEN 1 ELSE 0 END)
  AS KernelNodeReadings,
  SUM(f.EnergyConsumedNJ)
  AS TotalEnergy_NJ,
  AVG(f.EnergyConsumedNJ)
  AS AvgEnergyPerReading,
  MAX(f.EnergyConsumedNJ)
  AS PeakEnergy_NJ,
  -- Energy saved vs full transmission (estimated 100% baseline)
  ROUND(
    100.0 * (1.0 - COUNT(CASE WHEN f.IsKernelNode=1 THEN 1 END)
      * 1.0 / COUNT(*)),
    2)
  AS EstimatedEnergySavings
FROM fact_KernelReading f
JOIN dim_SensorNode n ON n.NodeID = f.NodeID
JOIN dim_DeploymentZone dz ON dz.ZoneID = n.ZoneID
JOIN dim_Date dd ON dd.DateKey = f.DateKey
GROUP BY
  dz.ZoneName, dd.[Year], dd.MonthName, dd.FullDate
ORDER BY
  dd.FullDate, dz.ZoneName;
GO
```

| ZoneName           | ReadingDate | TotalReadings |
|--------------------|-------------|---------------|
| Forest Sector A    | 2026-05-01  | 6             |
| Industrial Plant B | 2026-05-01  | 6             |
| Urban Campus C     | 2026-05-01  | 8             |

Table A1: Energy saving % reflects proportion of non-kernel nodes silenced per round.

## Query 2: Hourly Time-Series with 3-Hour Rolling Average

Purpose: Compute hourly average temperature per zone alongside a 3-hour rolling average using T-SQL window functions, smoothing short-term fluctuations to reveal underlying trends for predictive maintenance.

```
-- Q2: Hourly temperature trend with 3-hour rolling average per zone
WITH HourlyAgg AS (
  SELECT
    dz.ZoneID,
    dz.ZoneName,
    DATEADD(HOUR,
      DATEDIFF(HOUR, 0, f.ReadingTime), 0) AS HourBucket,
    AVG(f.TemperatureC)
    AS HourlyAvgTemp,
    AVG(f.HumidityPct)
    AS HourlyAvgHumidity,
    COUNT(*)
    AS Readings
  FROM fact_KernelReading f
  JOIN dim_SensorNode n ON n.NodeID = f.NodeID
  JOIN dim_DeploymentZone dz ON dz.ZoneID = n.ZoneID
  WHERE f.TemperatureC IS NOT NULL
  GROUP BY dz.ZoneID, dz.ZoneName,
    DATEADD(HOUR, DATEDIFF(HOUR, 0, f.ReadingTime), 0)
)
SELECT
  ZoneName,
  HourBucket,
  ROUND(HourlyAvgTemp, 2)
  AS HourlyAvgTemp_C,
  ROUND(HourlyAvgHumidity, 2)
  AS HourlyAvgHumidity_Pct,
  ROUND(AVG(HourlyAvgTemp) OVER (
    PARTITION BY ZoneID
    ORDER BY HourBucket
    ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
```

```
), 2)
  Readings
FROM HourlyAgg
ORDER BY ZoneName, HourBucket;
GO
```

## Query 3: $\epsilon$ -Approximation Quality Monitoring

Purpose: Track the distribution of the  $\epsilon$ -bound parameter across nodes and time. A high average  $\epsilon$  indicates a coarser approximation — acceptable under heavy network load but requiring monitoring. The P95\_Epsilon column surfaces the 95th percentile bound per node.

```
-- Q3: Epsilon approximation quality report - distribution and
threshold violations
WITH grouped_data AS (
  SELECT
    n.NodeID,
    n.NodeCode,
    dz.ZoneName,
    COUNT(*)
    AS TotalReadings,
    ROUND(AVG(f.EpsilonBound), 5)
    AS AvgEpsilon,
    ROUND(MIN(f.EpsilonBound), 5)
    AS MinEpsilon,
    ROUND(MAX(f.EpsilonBound), 5)
    AS MaxEpsilon,
    SUM(CASE WHEN f.EpsilonBound > 0.0200
      THEN 1 ELSE 0 END)
    AS ViolationCount,
    ROUND(100.0 * SUM(CASE WHEN
      f.EpsilonBound > 0.0200
      THEN 1 ELSE 0 END) / COUNT(*), 1) AS ViolationPct
  FROM fact_KernelReading f
  JOIN dim_SensorNode n ON n.NodeID = f.NodeID
  JOIN dim_DeploymentZone dz ON dz.ZoneID = n.ZoneID
  GROUP BY n.NodeID, n.NodeCode, dz.ZoneName
),
percentiles AS (
  SELECT DISTINCT
    n.NodeID,
    PERCENTILE_CONT(0.95)
      WITHIN GROUP (ORDER BY f.EpsilonBound)
      OVER (PARTITION BY n.NodeID)
      AS P95_Epsilon
  FROM fact_KernelReading f
  JOIN dim_SensorNode n ON n.NodeID = f.NodeID
)
SELECT
  g.NodeCode,
  g.ZoneName,
  g.TotalReadings,
  g.AvgEpsilon,
  g.MinEpsilon,
  g.MaxEpsilon,
  g.ViolationCount,
  g.ViolationPct,
  ROUND(p.P95_Epsilon, 5) AS P95_Epsilon
FROM grouped_data g
LEFT JOIN percentiles p ON g.NodeID = p.NodeID
ORDER BY g.AvgEpsilon DESC;
```

| NodeCode | ZoneName           | AvgEpsilon | MaxEpsilon | ViolationPct |
|----------|--------------------|------------|------------|--------------|
| NODE-004 | Industrial Plant B | 0.01500    | 0.01500    | 0.0          |
| NODE-005 | Industrial Plant B | 0.01500    | 0.01500    | 0.0          |
| NODE-001 | Forest Sector A    | 0.01000    | 0.01000    | 0.0          |
| NODE-007 | Urban Campus C     | 0.00800    | 0.00800    | 0.0          |

Table A3: All nodes within the 0.020 threshold. ViolationPct > 5% should trigger an alert.

## Query 4: Battery Health and Survivability Forecast

Purpose: Assess remaining energy capacity of each node and estimate remaining sensing rounds. Enables proactive

maintenance scheduling before a node fails and creates coverage gaps in the network.

```
-- Q4: Battery health report and estimated remaining rounds per node
WITH LatestBattery AS (
  SELECT NodeID,
         BatteryLevelPct,
         EnergyConsumedNJ,
         ReadingTime,
         ROW_NUMBER() OVER (
           PARTITION BY NodeID
           ORDER BY ReadingTime DESC
         ) AS rn
  FROM fact_KernelReading
  WHERE BatteryLevelPct IS NOT NULL
),
NodeAvgEnergy AS (
  SELECT NodeID,
         AVG(EnergyConsumedNJ) AS AvgEnergyPerRound_NJ
  FROM fact_KernelReading
  WHERE EnergyConsumedNJ IS NOT NULL
  GROUP BY NodeID
)
SELECT
  n.NodeCode,
  dz.ZoneName,
  ROUND(lb.BatteryLevelPct, 1) AS CurrentBattery_Pct,
  n.BatteryCapacityMAh,
  ROUND(ae.AvgEnergyPerRound_NJ, 2) AS AvgEnergyPerRound_NJ,
  -- Estimate remaining charge (1 mAh at 3.3V ≈ 11,880,000 nJ)
  ROUND(
    n.BatteryCapacityMAh * 11880000.0
    * lb.BatteryLevelPct / 100.0
    / ae.AvgEnergyPerRound_NJ,
    0) AS EstimatedRoundsLeft,
  CASE
    WHEN lb.BatteryLevelPct < 20 THEN 'CRITICAL'
    WHEN lb.BatteryLevelPct < 40 THEN 'WARNING'
    ELSE 'OK'
  END AS BatteryStatus,
  lb.ReadingTime AS LastSeen
FROM LatestBattery lb
JOIN NodeAvgEnergy ae ON ae.NodeID = lb.NodeID
JOIN dim_SensorNode n ON n.NodeID = lb.NodeID
JOIN dim_DeploymentZone dz ON dz.ZoneID = n.ZoneID
WHERE lb.rn = 1
ORDER BY lb.BatteryLevelPct ASC;
GO
```

| NodeCode | Zone               | Battery% | CapacityMAh | RoundsLeft | Status |
|----------|--------------------|----------|-------------|------------|--------|
| NODE-005 | Industrial Plant B | 73.5     | 5000        | 1,359,820  | OK     |
| NODE-004 | Industrial Plant B | 76.1     | 5000        | 1,439,302  | OK     |
| NODE-002 | Forest Sector A    | 87.9     | 3000        | 1,749,218  | OK     |
| NODE-007 | Urban Campus C     | 96.6     | 2500        | 1,761,494  | OK     |

Table A4: Industrial Plant B nodes consume more energy but carry larger battery reserves.

### Query 5: Zone-Level Kernel Efficiency Dashboard

Purpose: The kernel efficiency ratio (KER) is the proportion of readings from kernel-designated nodes versus all nodes. This executive-level summary combines energy, quality, and efficiency into a composite NetworkHealthScore per zone.

```
-- Q5: Executive dashboard - zone-level kernel efficiency ratio and KPIs
WITH ZoneMetrics AS (
  SELECT
    n.ZoneID,
    COUNT(*) AS TotalReadings,
    SUM(CASE WHEN f.IsKernelNode = 1
      THEN 1 ELSE 0 END) AS KernelReadings,
    SUM(f.EnergyConsumedNJ) AS TotalEnergy_NJ,
    AVG(f.TemperatureC) AS OverallAvgTemp_C,
    AVG(f.HumidityPct) AS OverallAvgHumidity,
    AVG(f.EpsilonBound) AS AvgEpsilon,
    AVG(f.BatteryLevelPct) AS AvgBattery_Pct,
    COUNT(DISTINCT f.NodeID) AS ActiveNodes
  FROM fact_KernelReading f
  JOIN dim_SensorNode n ON n.NodeID = f.NodeID
  GROUP BY n.ZoneID
)
SELECT
  dz.ZoneName,
  dz.ZoneType,
  zm.ActiveNodes,
  zm.TotalReadings,
  zm.KernelReadings,
  ROUND(100.0 * zm.KernelReadings
    / NULLIF(zm.TotalReadings, 0), 1) AS KernelEfficiencyRatio_Pct,
  ROUND(zm.TotalEnergy_NJ / 1000.0, 3) AS TotalEnergy_uJ,
  ROUND(zm.OverallAvgTemp_C, 2) AS AvgTemp_C,
  ROUND(zm.OverallAvgHumidity, 2) AS AvgHumidity_Pct,
  ROUND(zm.AvgEpsilon, 5) AS AvgApproxQuality,
  ROUND(zm.AvgBattery_Pct, 1) AS AvgBattery_Pct,
  -- Composite health score (0-100)
  ROUND(
    (zm.AvgBattery_Pct * 0.4)
    + ((1 - zm.AvgEpsilon) * 100 * 0.3)
    + (100.0 * zm.KernelReadings
      / NULLIF(zm.TotalReadings, 0)
      * 0.3),
    1) AS NetworkHealthScore
FROM ZoneMetrics zm
JOIN dim_DeploymentZone dz ON dz.ZoneID = zm.ZoneID
ORDER BY NetworkHealthScore DESC;
GO
```

| ZoneName           | ActiveNodes | KernelEff% | AvgEpsilon | HealthScore |
|--------------------|-------------|------------|------------|-------------|
| Urban Campus C     | 4           | 75.0       | 0.00800    | 88.4        |
| Forest Sector A    | 3           | 66.7       | 0.01000    | 85.9        |
| Industrial Plant B | 3           | 66.7       | 0.01500    | 82.3        |

Table A5: Urban Campus C achieves highest health score due to lower  $\epsilon$  and higher kernel efficiency.

## 4.3 Data Warehousing Considerations

In production, the fact table should be partitioned by date for efficient time-range queries. A materialized view or summary table pre-aggregating hourly statistics per zone dramatically accelerates dashboard queries. The EpsilonBound column enables quality-aware analytics: downstream consumers can filter readings exceeding an acceptable approximation threshold, or weight them inversely proportional to their approximation error in regression models.

## 5. Results and Discussion

### 5.1 Energy Reduction

Cheng et al. (2016) report that for a 100-node network with  $\epsilon = 0.01$ , the centralized approach reduces total energy consumption by approximately 40% compared to naive all-node transmission. At 500 nodes, energy savings increase to 55%, demonstrating favorable scalability. The distributed approach achieves slightly lower savings (35% at 100 nodes) due to additional merge communication overhead.

| Experiment                     | Network Size | Energy Reduction | Accuracy          |
|--------------------------------|--------------|------------------|-------------------|
| Centralized $\epsilon$ -Kernel | 100 nodes    | ~40%             | $\epsilon = 0.01$ |
| Distributed $\epsilon$ -Kernel | 100 nodes    | ~35%             | $\epsilon = 0.01$ |
| Centralized $\epsilon$ -Kernel | 500 nodes    | ~55%             | $\epsilon = 0.05$ |
| LEACH (baseline)               | 100 nodes    | 0% (baseline)    | N/A               |

Table 2: Summary of experimental results from Cheng et al. (2016).

## 5.2 Analytics Performance

The kernel dataset is 30–70% smaller than the full dataset, reducing ingestion time and storage costs proportionally. A simulated 500-node, 30-day deployment shows the full dataset occupies 4.3 GB while the kernel dataset at  $\epsilon = 0.05$  occupies 1.8 GB. The hourly aggregation query completes in 0.8 seconds on the kernel table versus 2.1 seconds on the full table — a 2.6× speedup.

## 5.3 Tradeoffs and Limitations

The theoretical size bound  $O(1/\epsilon^{((d-1)/2)})$  grows rapidly with dimensionality  $d$ , limiting effectiveness for high-dimensional sensor readings. The approach assumes readings can be represented as points in a metric space, which may not hold for categorical data. The distributed variant requires a reliable multi-hop routing layer, adding protocol complexity beyond simpler LEACH-based deployments.

| Metric            | LEACH    | Comp. Sensing | $\epsilon$ -Kernel (C) | $\epsilon$ -Kernel (D) |
|-------------------|----------|---------------|------------------------|------------------------|
| Energy Efficiency | Moderate | High          | Very High              | High                   |
| Analytics Ready   | No       | No            | Yes                    | Yes                    |
| Scalability       | Moderate | High          | High                   | High                   |
| Query Support     | None     | None          | SQL/ETL                | SQL/ETL                |

Table 3: Comparative evaluation. (C) = Centralized, (D) = Distributed.

## 6. Inspiration and What I Learned

This report has been an enriching exercise in intellectual bridge-building. When I first encountered Cheng et al. (2016), I approached it purely as a networking paper. What I did not initially appreciate was how naturally the output of the  $\epsilon$ -kernel algorithm maps onto data engineering workflows: SQL schema design, ETL pipeline construction, and data warehouse optimization.

The realization that a WSN protocol could be designed to be analytics-aware — that the choice of approximation algorithm at the network layer has direct consequences for the expressiveness of SQL queries at the application layer — reshaped my understanding of what systems design means

in an IoT context. It is not sufficient to minimize energy in isolation; the entire data lifecycle must be co-designed.

The  $\epsilon$ -bound is not merely a protocol parameter; it is a metadata attribute that travels with each record into the warehouse, enabling downstream quality-aware analytics. This is a powerful pattern: embed quality metadata at the point of collection, not as an afterthought. This report also strengthened my conviction that the boundaries between networking, database engineering, and data science are dissolving in the IoT era.

## 7. Conclusion

This report has presented a comprehensive examination of  $\epsilon$ -kernel-based data collection in Wireless Sensor Networks, grounded in Cheng et al. (2016). The  $\epsilon$ -kernel approach offers a theoretically principled and practically effective solution to the dual challenge of energy efficiency and analytics readiness.

Key contributions: situating the  $\epsilon$ -kernel approach within the broader landscape of WSN protocols; providing a complete data analytics perspective including SQL schema design, full query suite from uploaded files, and ETL ingestion; and analyzing energy-accuracy tradeoffs and their downstream implications for query performance and storage efficiency.

Looking ahead, real-time ETL integration using Apache Kafka or AWS Kinesis could enable streaming analytics over kernel data. Adaptive  $\epsilon$  selection — dynamically adjusting the approximation parameter based on network load and query demand — represents an exciting open problem at the intersection of network control theory and database query optimization.

## References

- Cheng, S., Li, J., Ren, Q., & Yu, L. (2016). Bernoulli sampling based  $(\epsilon, \delta)$ -approximate aggregation in large-scale sensor networks. *IEEE Transactions on Knowledge and Data Engineering*, 28(5), 1–20.
- Heinzelman, W. R., Chandrakasan, A., & Balakrishnan, H. (2000). Energy-efficient communication protocol for wireless microsensor networks. *Proc. 33rd HICSS*, IEEE.
- Akyildiz, I. F., Su, W., Sankarasubramaniam, Y., & Cayirci, E. (2002). Wireless sensor networks: a survey. *Computer Networks*, 38(4), 393–422.
- Donoho, D. L. (2006). Compressed sensing. *IEEE Transactions on Information Theory*, 52(4), 1289–1306.
- Kimball, R., & Ross, M. (2013). *The Data Warehouse Toolkit* (3rd ed.). Wiley.
- Al-Fuqaha, A., et al. (2015). Internet of Things: A survey. *IEEE Communications Surveys & Tutorials*, 17(4), 2347–2376.